Thread synchronization

05 May 2024 10:39

Synchronization upon termination: join() method

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class Ecrivain here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Ecrivain extends Thread
{
   private String text;
   public Ecrivain(String t){
      text = t;
   }

   @Override
   public void run(){
      int i, j;
      for(i = 0; i < 10; i++){
         for(j = 0; j < text.length()-1; j++){
            System.out.print(text.substring(j, j+1));
            try{
               sleep((long)(Math.random() * 100));
            }
            catch(InterruptedException ie){}
         }
         System.out.print(text.substring(j, j+1)+"\n");
      }
      System.out.print("\necrivane " + text + " gottta");
   }
}
```

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class TestEcrivain here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestEcrivain
{
   public static void main(String args[]){
      Ecrivain ob1 = new Ecrivain("ABC");
      Ecrivain ob2 = new Ecrivain("XYZ");
      ob1.start();
      ob2.start();
   }
}
```

Output:

```
AXYZXBCAYZXBYCAZXYBZXYCABZXYCAZXYBCABCABCAZXBYCA
ZXYZXYBZ
ecrivane XYZ gotttaCABC
ecrivane ABC gottta
```

Output is unpredictable because the threads were not synchronized.

There is concurrence between both writers and that is the reason why we obtained the above execution.
Using the join() method of Thread class we can synchronize the two threads. ob2 will be launched after the termination of ob1.

The code presented after join() will be executed after the end of the execution of the code located before join().

```
package ThreadsForSubhalakshmi;

/**
 * Write a description of class Ecrivain here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Ecrivain extends Thread
{
   private String text;
   public Ecrivain(String t){
      text = t;
   }

   @Override
   public void run(){
      int i, j;
      for(i = 0; i < 10; i++){
         for(j = 0; j < text.length()-1; j++){
            System.out.print(text.substring(j, j+1));
            try{
               sleep((long)(Math.random() * 1000));
            }
            catch(InterruptedException ie){}
         }
         System.out.print(text.substring(j, j+1)+"\n");
      }
      System.out.printf("%s %s %s","\necrivane ",text, " gottta");
   }
}
package ThreadsForSubhalakshmi;


/**
 * Write a description of class TestEcrivain here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestEcrivain
{
   public static void main(String args[]){
      System.out.print("\f");
      Ecrivain ob1 = new Ecrivain("ABC");
      Ecrivain ob2 = new Ecrivain("XYZ");
      ob1.start();
      try
      {
         ob1.join();
      }
      catch (InterruptedException ie)
      {
         System.out.print(ie.getMessage()+"\n");
         System.exit(1);
      }
      ob2.start();
   }
```

}

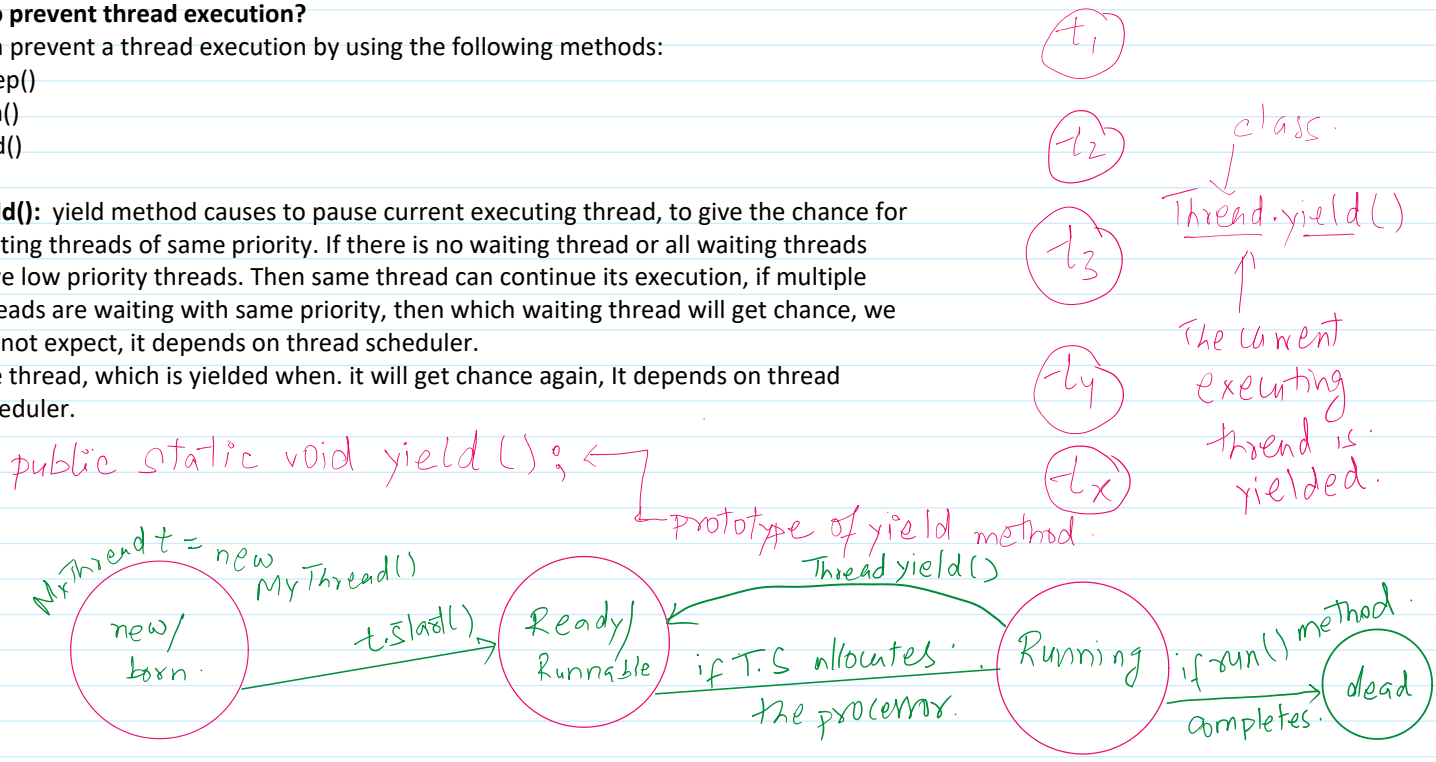
Output:
ABC
ABC
ABC
ABC
ABC
ABC
ABC
ABC
ABC
ABC

ecrivane ABC gotttaXYZ
XYZ
XYZ
XYZ
XYZ
XYZ
XYZ
XYZ
XYZ
XYZ

ecrivane XYZ gottta

How to prevent thread execution?
We can prevent a thread execution by using the following methods:
3. sleep()
2. join()
1. yield()

1. **yield():** yield method causes to pause current executing thread, to give the chance for waiting threads of same priority. If there is no waiting thread or all waiting threads have low priority threads. Then same thread can continue its execution, if multiple threads are waiting with same priority, then which waiting thread will get chance, we cannot expect, it depends on thread scheduler.
   The thread, which is yielded when. it will get chance again, It depends on thread scheduler.



package ThreadsForSubhalakshmi;

package ThreadsForSubhalakshmi;

/**

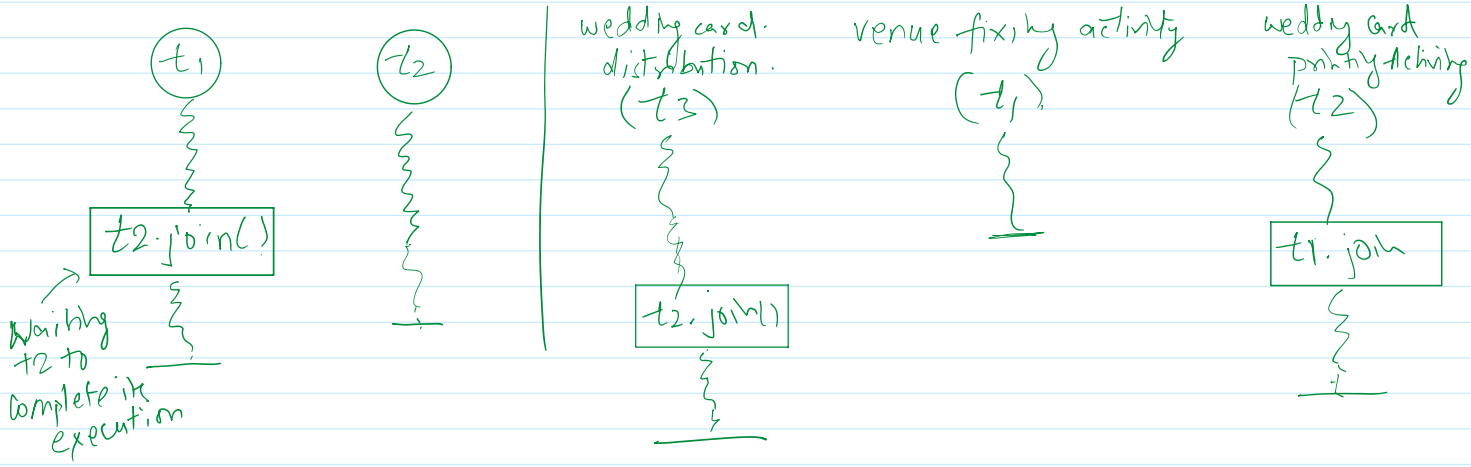
```
/**
 * Write a description of class MyThread here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyThread extends Thread
{
   public void run(){
      int i;
      for(i = 0; i < 10; i++){
         System.out.print("\nChild Thread");
         Thread.yield();  //1
      }
   }
}
```

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class ThreadYieldDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ThreadYieldDemo
{
   public static void main(String args[]){
      MyThread t = new MyThread();
      t.start();
      t.setPriority(Thread.MAX_PRIORITY);
      int i;
      for(i = 0; i < 10; i++){
         System.out.print("\nMain Thread");
      }
   }
}
```

In the above program, if we are commenting line 1 then both threads will be executed simultaneously and we can't expect, which thread can complete first. If we are not commenting line 1, then child thread will always call yield() method because of that main thread will get a chance more number of times and the chance of completing the main thread first is high.

Note: Some platform won't provide proper support for yeild() method.



If a thread wants to wait until completing. some other threads, then we should go for join() Method. For exa1:1mple, if a thread T1 wants to wait until completing T2 then T1 has to call T2.join(); if T1 executes T2.Join() then immediately T1 will be entered into waiting state until T2 completes. Ones T2 completes then T1 can continue its execution.

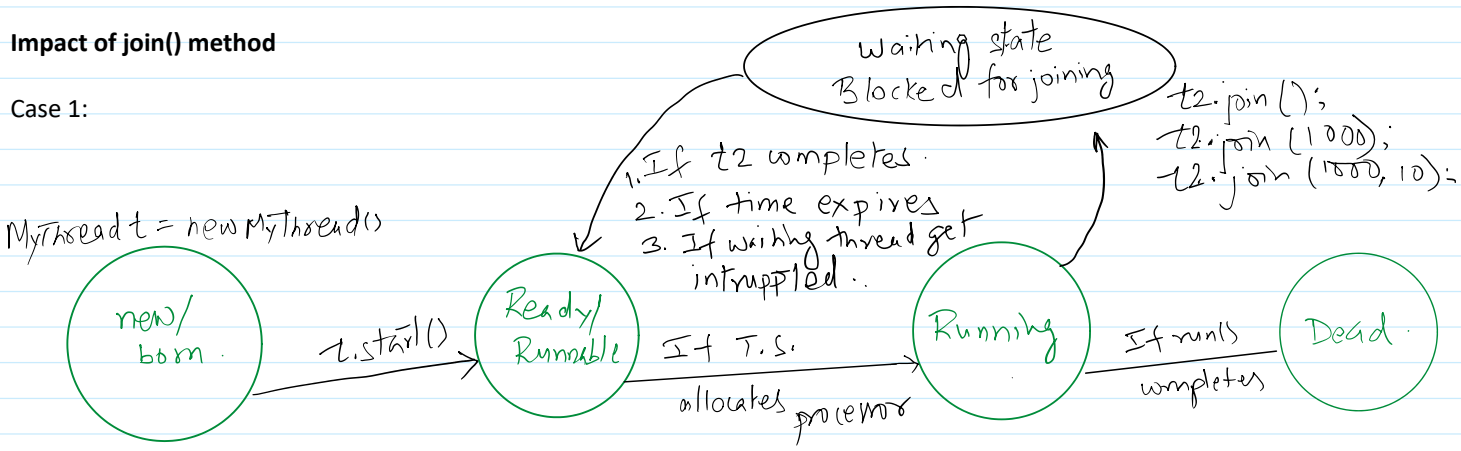
public final void join() throws InterruptedException
public final void join(long ms) throws InterruptedException
public final void join(long ms, int ns) throws InterruptedException

Note: Every join() throws InterruptedException which is checked Exception. Hence compulsory we should handle this exception, either by using try…catch or by using throws keyword, otherwise we will get compile time error.

Note: Every join() throws InterruptedException which is checked Exception. Hence compulsory we should handle this exception, either by using try…catch or by using throws keyword, otherwise we will get compile time error.

Impact of join() method

Case 1:



Waiting of main Thread until completion of child thread.

```
public class MyThread1 extends Thread
{
   public void run(){
      for(int i = 0; i < 10; i++){
         System.out.print("\nSita Thread");
         try{
            Thread.sleep(2000);
         }
         catch(InterruptedException e){}
      }
   }
}

package ThreadsForSubhalakshmi;


/**
 * Write a description of class ThreadJoinDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ThreadJoinDemo
{
   public static void main(String args[]) throws
InterruptedException{
      MyThread1 t = new MyThread1();
      t.start();
      //t.join(5000, 100); //1
     t.join();
     for(int i = 0; i < 10; i++){
         System.out.print("\nRam Thread");
      }
   }
}
```

if we comment the line, then both main and child thread execute simultaneously and we can't expect exact output.

Otherwise main thread calls join() method on child thread object. Hence main thread wait until child thread complete its execution. check the output. 'Sita thread' 10 times and 'Ram thread' 10 times.

Case 2: Waiting of child thread until completing main thread

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class MyThread2 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyThread2 extends Thread
{
   static Thread init;

   public void run(){
      try
      {
         init.join();
      }
      catch (InterruptedException ie)
      {
         ie.printStackTrace();
      }
      for(int i = 0; i < 10; i++){
         System.out.print("\nChild thread");
      }
   }
}
```

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class Thread2JoinDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Thread2JoinDemo
{
   public static void main(String args[]){
      MyThread2.init = Thread.currentThread();
      MyThread2 t = new MyThread2();
      t.start();
      for(int i = 0; i < 10; i++){
         System.out.print("\nMain Thread");
         try
         {
            Thread.sleep(2000);
         }
         catch (InterruptedException ie)
         {
            ie.printStackTrace();
         }
      }
   }
}
```

Output:

```
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Main Thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
Child thread
```

In the above example, child thread calls join method on Main. thread object. Hence, child thread has to wait until completing the main thread. In this case, output is "Main Thread" after that "Child thread".

Case-3
Deadlock situation: ((loop holes of join())

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class MyThread2 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyThread2 extends Thread
{
   static Thread init;

   public void run(){
      try
      {
         init.join();//Deadlock, because child thread has called the join method on main thread.
      }
      catch (InterruptedException ie)
      {
         ie.printStackTrace();
      }
      for(int i = 0; i < 10; i++){
         System.out.print("\nChild thread");
```

```
    }
  }
}
```

If main thread calls the join() method on child thread object and if child thread calls join() method on main thread object, then both thread will wait forever program will be struck. This is something called deadlock situation.

Case- 4 (loop holes of join())

If your thread calls join() method. on the same thread itself, then the program will be struck. This is something like deadlock situation. In this case thread has to wait infinite amount of time.

```
public class DeadLock
{
  public static void main(String arg[]){
    try
    {
      Thread.currentThread().join();
    }
    catch (InterruptedException ie)
    {
      ie.printStackTrace();
    }
  }
}
```

sleep() method

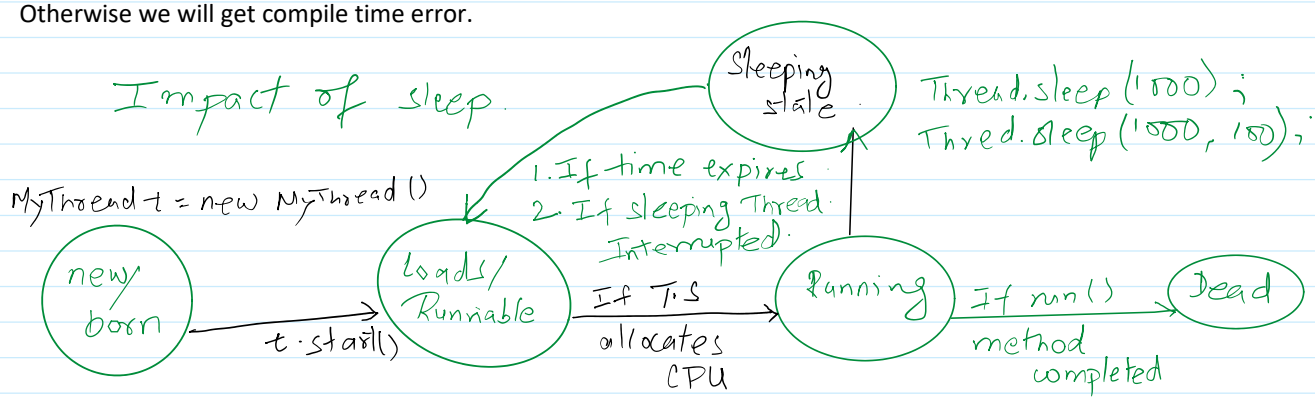
If a thread don't want to perform any operation for a particular amount of time, then we should go for sleep method.

pubic static void sleep(long ms) throws InterruptedException; //signature of the sleep method.

pubic static void sleep(long ms, int ns) throws IntrerruptedException; //signature of the sleep method.

Note:- Every sleep() method throws interrupted exception, which is checked Hence, whenever we are using sleep method compulsory, we should handle interrupted exception, either by Try... Catch or by throws keyword. Otherwise we will get compile time error.

Impact of sleep.



How a thread can interrupt another thread?

public void interrupt();

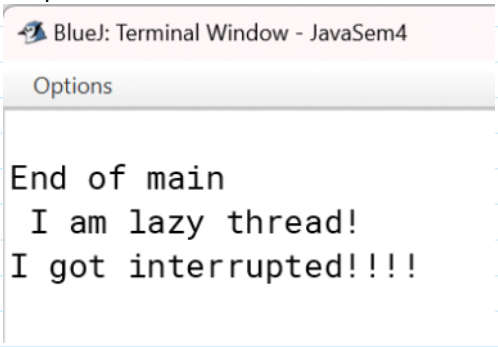
A thread can interrupt a sleeping Thread or waiting thread by using the interrupt() method Thread class

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class MyThread3 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyThread3 extends Thread
{
   @Override
   public void run(){
      try{
         for(int i = 0; i < 10; i++){
            System.out.print("\n I am lazy thread!");
            Thread.sleep(1000);
         }
      }
      catch(InterruptedException ie){
         System.out.print("\nI got interrupted!!!!");
      }
   }
}

public class ThreadInterruptedDemo
{
   public static void main(String args[]){
      MyThread3 t = new MyThread3();
      t.start();
      t.interrupt();
      System.out.print("\nEnd of main");
   }
}
```

Output:



Note: Whenever we are calling interrupt() method if. the target thread is not in sleeping state or in waiting thread. then there is no impact of interrupt call. interrupt call can be weighted till target thread entered into the sleeping or waiting state. If the target thread entered into sleeping or waiting thread then immediately interrupt call with the interrupt target thread. If the target thread never entered into sleeping or waiting state in its lifetime, then there is no impact of interrupt call. This is. the only case where interrupt call will be wasted.

Comparison table of yield(), join() and sleep() methods.

Property	yield()	join()	sleep()
Purpose	If thread wants to pause its execution to give the chance for	If a thread want to wait until the completion of some other thread,	If a thread don't want to perform any operation for a particular

	remaining threads of same priority then. we should go for yield method.	then we should go for join method.	amount of time, then we should go for a sleep method.
Overloaded	No	Yes	Yes
Is it final	No	Yes	No
Throws InterruptedException	No	Yes	Yes
Static	Yes	No	yes